MULTI-CITY PROBABILISTIC WEATHER FORECASTING USING TEMPORAL FUSION TRANSFORMERS AND EXPLAINABLE AI

A comparison of pooled and single- city models across five climates

By

Tharun Bisai

Student ID: A00066558

Submitted to

The University of Roehampton

In partial fulfilment of the requirements

For the degree of

Master of Science

in

Data Science

Supervisor: Dr. Sayed Pouria Talebi

Submission Date: August 2026

DECLARATION

I hereby certify that this report constitutes my own work, that where the language of others is used, quotation marks so indicate, and that appropriate credit is given where I have used the language, ideas, expressions, or writings of others.

I declare that this report describes the original work that has not been previously presented for the award of any other degree of any other institution.

Name: Tharun Bisai

Date: 11 August 2026

Signature: Tharun

ACKNOWLEDGEMENTS

I would like to thank my supervisor, Dr. Sayed Pouria Talebi, for his guidance throughout this project. His questions consistently identified the weakest parts of the work rather than the most visible ones. Two of the findings reported here exist because of that: the request for standard baselines produced the comparison that frames Chapter 4, and the observation that only one model had been tuned led to the correction that became the project's central methodological result.

I am also grateful to the staff of the Department of Computer Science at the University of Roehampton for their teaching over the course of the programme, and to my family and friends for their support and patience while this work was carried out.

ABSTRACT

Weather forecasting operates by using numerical models which solve the equations that govern the atmosphere. However, recent studies have demonstrated that data-driven models, which are trained on historical reanalysis data, can match these models at a much lower computational cost. Since a great deal of the research in this field has been based on a single location, a practical issue remains for anyone who wants to put such a system into use: it is not clear whether a single model developed for several cities can be used to serve all of them, or whether each location will need its own model.

The study makes use of ten years' worth of hourly observations from five cities that were selected because of their climatic variety—Jena, London, New York, Sydney and Tokyo. The data set includes 438,360 full hourly records covering 23 meteorological variables. Two deep learning models, a long short-term memory network and a Temporal Fusion Transformer, are compared with one another and with four statistical and machine learning baseline methods in the context of a 24-hour forecasting task, employing a train, validation and test split that follows chronological order in order to prevent look-ahead bias.

The following four points can be identified. First, a single model that has been trained across all five cities gives a mean absolute error of 1.268 degrees Celsius, as compared with 1.272 degrees for five models each trained in their respective cities, a difference of just four thousandths of a degree. It follows that pooling incurs no loss in accuracy while cutting the number of models from five to one. Second, when all the methods are given the same hyperparameter search, the Temporal Fusion Transformer, gradient boosting and the long short-term memory network all end up within 0.02 degrees of each other, the Transformer achieving 1.247 degrees, gradient boosting 1.263 degrees and the long short-term memory network 1.266 degrees respectively. A previous comparison in which only the transformer had been tuned had inflated its advantage by about a factor of seven. Training the recurrent network to convergence eliminates the remaining gap, and it is therefore concluded that no architecture is meaningfully superior to the others and that the variation caused by the training protocol is of the same magnitude as the variation between the different architectures. Third, the prediction intervals produced by conformal prediction have empirical coverage between 90 and 93 per cent at each of the 24 forecast horizons,

increasing from plus or minus 1.26 degrees after one hour to plus or minus 4.81 degrees after twenty-four hours. Fourth, two separate interpretability techniques agree that the most recent temperature history is what dominates the forecast, while the transformer's attention focuses on the previous half day even though it is given a full week's context.

The study provides an empirical response to the pooling question, shows that tuning parity is more important than architectural choice with regard to this task, and includes a browser-based tool that carries out neural inference on the client device together with calibrated uncertainty intervals.

Key words: Weather Forecasting, Temporal Fusion Transformer, Long Short-Term Memory, Global and Local Forecasting Models, Conformal Prediction, Explainable AI, ONNX

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

CHAPTER 1 INTRODUCTION

Accurate short-term forecasting of temperature is essential for decisions in the fields of energy, agriculture and public health, electricity demand forecasting in particular relying on it (Hong and Fan, 2016). The traditional method is numerical weather prediction, which involves applying the equations of atmospheric physics on supercomputers. Although its accuracy has improved steadily over the decades with a gain of about one day of lead time each decade (Bauer, Thorpe and Brunet, 2015), it is computationally expensive and is produced centrally at fixed intervals.

Another line of research regards forecasting as a supervised learning task based on historical observations. Large-scale systems recently trained on decades of global reanalysis, for example Pangu-Weather (Bi et al., 2023) and GraphCast (Lam et al., 2023), have shown that this method is able to match or outperform operational numerical models on several criteria while being considerably faster once they have been trained. This outcome has turned data-driven forecasting into a legitimate area of study rather than just a speculative idea.

The project looks at a more specific question. In the literature on forecasting, local models—which are fitted to a single series—are distinguished from global models which are fitted to many series (Januschowski et al., 2020), and so an organisation running in several cities must make a choice between these two types. A pooled model makes use of a larger amount of data but must deal with conflicting seasonal patterns, whereas a per-city model focuses on only a portion of the data. Montero-Manso and Hyndman (2021) claim that global models do not have to be restrictive even when applied to heterogeneous series, although their evidence is based on many short and comparable series rather than a small number of long ones which are climatically different.

1.1 DESCRIPTION OF THE PROBLEM, THE CONTEXT AND THE MOTIVATION

The issue under consideration is whether using weather data from multiple cities gives better results for 24-hour temperature forecasting when compared to training on just one city all by itself, and whether the extra complexity involved in employing an attention-based approach such as the Temporal Fusion Transformer (Lim et al., 2021) is worth it when compared with simpler methods. This question is worthwhile raising since recent studies have on several occasions found that carefully tuned simple

models perform as well as more complicated architectures on forecasting tasks (Zeng et al., 2023) and on tabular data (Grinsztajn, Oyallon and Varoquaux, 2022).

The reason why five cities were chosen is that they have different climates: Jena, which has a continental climate; London, which is maritime; New York, which exhibits large continental temperature variations; Tokyo, with a humid subtropical climate; and Sydney, located in the southern hemisphere. Sydney is the important one to include since its seasons are reversed, and so the pooled model must deal with the fact that January is cold in four of the cities but warm in the fifth. The daily temperatures in the four northern cities correlate between +0.81 and +0.90 with one another, whereas Sydney's correlations with all of them range from -0.76 to -0.82 (Figure 1).

The question of pooling is therefore truly open since if one model can absorb five conflicting seasonal regimes without any loss then this is a useful and non-obvious result, but if it cannot then the cost of specialisation can be measured.

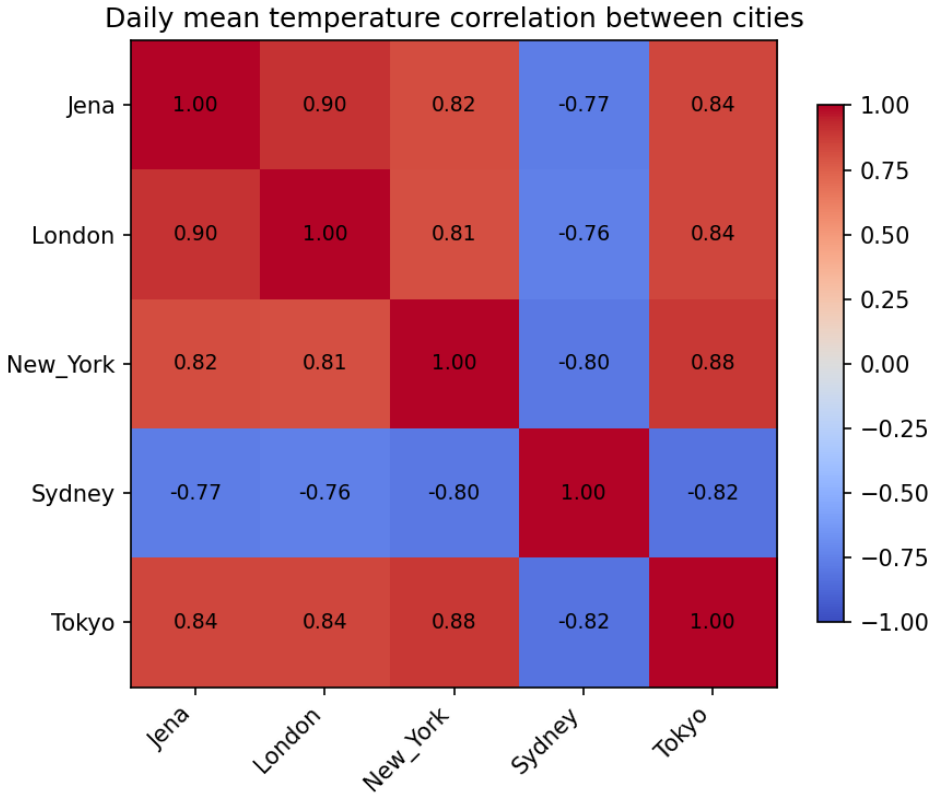

Figure 1: The daily mean temperature of the five cities is shown, with the four northern ones showing a similar trend while Sydney is negatively correlated with each of them.

1.2 OBJECTIVES

The project had four aims, each stated in the form of a research question.

The first research question investigates whether a Temporal Fusion Transformer (Lim et al., 2021) can beat a long short-term memory network (Hochreiter and Schmidhuber, 1997) when it comes to 24-hour forecasting for the five cities, thereby examining if attention-based architectures are justified in terms of their complexity in this case.

The central issue is and the one which has the most clear practical implication is whether models that have been trained on individual cities perform better than a single model that has been trained across all five.

The question raised by RQ3 is which of the input variables have an effect on the forecasts, since this is important for building trust and for determining which measurements are worth taking.

Question RQ4 is concerned with whether conformal prediction (Vovk, Gammerman and Shafer, 2005) is capable of providing calibrated intervals over the entire range, since a point forecast lacking an honest error interval has only limited practical value.

Another objective was to produce a working artifact, one that demonstrated the deployment rather than just leaving the model in the form of a notebook output. This artifact has now been deployed and is available to the public at tharun2431.github.io/weather-forecasting-msc.

1.3 METHODOLOGY

The method adopted is empirical and comparative. Hourly observations from ten years for each city were combined to form a consistent set of features and were then divided in chronological order into training, validation and test sections in a ratio of 70:15:15. It is necessary to carry out a chronological division since carrying out random shuffling would allow the model to learn from observations that came after the ones it is predicting, thus leading to an overestimated level of accuracy.

Each model is given a context window of 168 hours and then forecasts the next 24 hours. The tuning was done only on the validation subset, and during the development process the test subset was not referred to at all, so the figures cited give an honest estimate of performance on something never seen before.

It had been deliberately decided that the same amount of tuning should be applied to each method; in an earlier version only the transformer was tuned and this led to an overstatement of its advantages, an experience which is described in Chapter 4.

1.4 LEGAL, SOCIAL, ETHICAL AND PROFESSIONAL CONSIDERATIONS

The data consist of hourly reanalysis figures obtained from the Open-Meteo API, which in turn is based on ERA5 (Hersbach et al., 2020) and can be used freely for research purposes. The data set gives a description of atmospheric conditions and includes no personal information; accordingly, the project presents no issues under the General Data Protection Regulation and therefore did not require any ethical approval.

There are two points that are worth considering. Giving a forecast without also providing an error estimate can lead to overconfidence, especially when the method's reasoning is not clear; this is one of the reasons why the conformal prediction approach was developed, and the artifact includes the interval together with each forecast. Secondly, these models produce temperature forecasts for five cities over a fixed period, are not operational systems, and the artifact makes it clear that this is a research demonstration.

The social aspect is determined by the locations that were studied. These five cities are all large, wealthy and well equipped with observational instruments, and four of them are situated in the northern mid-latitudes. Weather forecasting is most important in areas where people's livelihoods depend directly on it and where the meteorological infrastructure is most limited, a situation which applies to almost none of the places included in this study. The fact that a pooled model can be shared among these five well-observed cities does not prove that it would work in a data-sparse area, and it would therefore overstate the benefits to the populations who are least served by current forecasting systems. This problem is exacerbated by the reanalysis source since its accuracy varies from place to place and is highest in areas where observations are most numerous. Thus the finding that pooling incurs no cost is a conclusion specific to these five cities, and the broader assertion it implies—that a single model can be used in many different locations—would have to be tested in a region where the issue is actually relevant.

Another social issue is access. Since the artefact operates completely within a web browser and carries out its inference on the user's own device, it needs no account, does not transmit any personal data and has no per-query charge. The fact that it is this way means that the demonstration can be used on a low-specification phone, which may seem like a small advantage but is intentional.

The energy required to train deep networks is also a factor, which is ironic in the context of a project concerned with weather. The training was kept at a proportional level by using modest model sizes, limiting the searches, and ensuring that the models were small enough to allow inference to be carried out on a mobile browser.

1.5 BACKGROUND

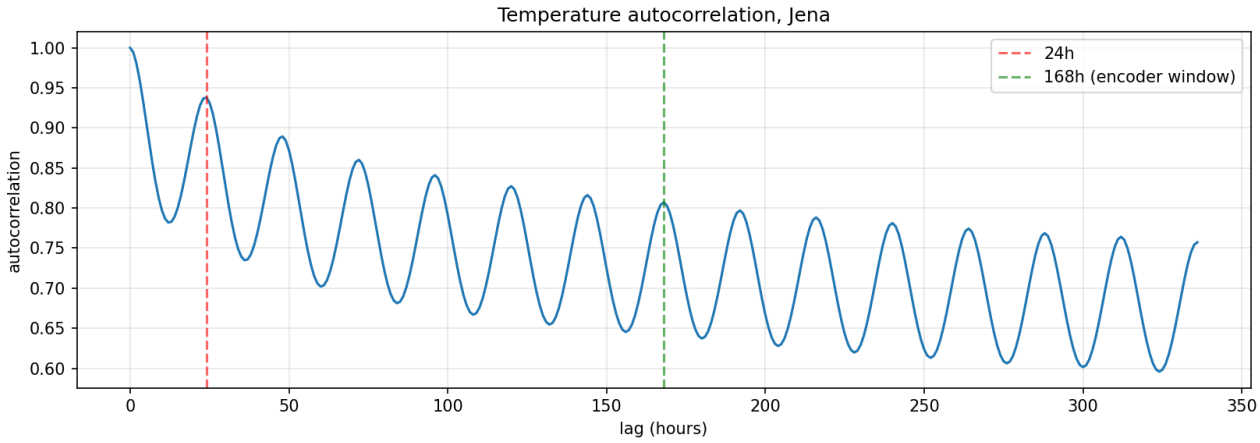

Figure 2: Shows the autocorrelation of the hourly temperature at Jena; correlation is observed for several days with clear peaks each 24 hours, which supports the use of a 168-hour context window.

Temperature shows autocorrelation at several timescales (Box and Jenkins, 1970): a daily cycle from solar heating, an annual cycle from axial tilt, and synoptic variation over days. Traditional methods model these directly using differencing and seasonal terms. Machine learning methods learn from engineered lags and cyclical encodings, while deep sequence models learn patterns from the raw data. The 168-hour window used here is based on autocorrelation analysis, which indicated correlation lasting for days with distinct 24-hour peaks (Figure 2).

1.6 STRUCTURE OF REPORT

Chapter 2 reviews the existing literature on forecasting, uncertainty quantification, and interpretability. It also surveys the technologies used. Chapter 3 describes the implementation. Chapter 4 presents the evaluation, addressing each research question one at a time. Chapter 5 wraps up, identifies future work, and reflects on how the project was carried out.

CHAPTER 2 LITERATURE – TECHNOLOGY REVIEW

This chapter goes over the work this project builds upon. The first section discusses forecasting methods. It ranges from classical statistical techniques to recurrent and attention-based architectures and includes recent large-scale weather models. It also covers the literature on global versus local models, which is central to this project. Additionally, it addresses uncertainty quantification and interpretability, as both are key topics here. The second section explains the technologies used and the reasons for their selection.

2.1 LITERATURE REVIEW

Statistical time series forecasting has a long history. The autoregressive integrated moving average framework established by Box and Jenkins (1970) is still a standard reference, and its seasonal extension explicitly addresses periodic structures. The M4 forecasting competition reported by Makridakis, Spiliotis, and Assimakopoulos (2020) found that purely machine learning methods often struggled to outperform well-defined statistical baselines. This finding led to the decision to include statistical baselines in this project instead of only comparing deep models to each other.

Deep sequence modeling for forecasting began with recurrent architectures. The long short-term memory network developed by Hochreiter and Schmidhuber (1997) introduced gating that helped solve the vanishing gradient problem, making it feasible to learn from long sequences. Recurrent models were dominant in sequence learning until the transformer architecture introduced by Vaswani et al. (2017) removed recurrence in favor of self-attention. This shift allowed for parallel computation and better modeling of long-range dependencies.

Applying transformers to forecasting required adjustments. The Temporal Fusion Transformer developed by Lim, Arik, Loeff, and Pfister (2021) is the architecture examined here. It combines recurrent layers for local processing with attention for long-range structures. It also includes variable selection networks that learn how much weight to give each input at every step. It clearly distinguishes between covariates known in advance, such as calendar features, and those only available up to the forecast origin. This design helps prevent information leakage. Its quantile loss creates prediction intervals directly, and its variable selection weights aim to be interpretable. Related architectures include DeepAR (Salinas et al., 2020), which models

probabilistic forecasts in an autoregressive manner, and N-BEATS (Oreshkin et al., 2020) along with its successor NHiTS (Challu et al., 2023), which use deep layers of fully connected blocks.

The assumption that deep architectures always perform better than simpler methods has been questioned. Zeng et al. (2023) demonstrated that a single linear layer matched or outperformed several transformer variants on common forecasting benchmarks. This finding calls into question how much of the reported improvements were due to architecture alone. Grinsztajn, Oyallon, and Varoquaux (2022) showed that gradient-boosted trees often do better than deep learning on many tabular problems. Both results anticipate outcomes reported in Chapter 4, where a fine-tuned gradient boosting model comes within two hundredths of a degree of a fine-tuned transformer.

At the largest scale, data-driven weather prediction has progressed quickly. Pangu-Weather (Bi et al., 2023) and GraphCast (Lam et al., 2023) draw on decades of global reanalysis data and have demonstrated skill comparable to operational numerical systems, all while using far less computational power. These systems operate on a scale beyond this project, but they demonstrate that learning atmospheric dynamics from data is possible. This work builds on that foundation at the city level.

The choice between using one model for multiple series or one model for each series is a well-discussed topic in forecasting literature, labeled as global and local models. Januschowski et al. (2020) emphasize this distinction as a key way to classify forecasting methods, noting that it bridges the common divide between statistical and machine learning approaches. Montero-Manso and Hyndman (2021) develop the theory behind this, asserting that a global model fitted across multiple series is not necessarily limiting, even when the series are diverse. A sufficiently flexible global model can capture local behaviors through its parameters. They demonstrate that global models can perform as well as or better than local ones without requiring the series to be alike. This idea is counter-intuitive and directly significant to the present work.

This project specifically addresses that gap. The theoretical argument for global models typically concerns groups of related series in theory. However, the empirical evidence often comes from competition datasets with many short, similar series. The

situation examined here is different. It involves a small number of long series from locations with opposing seasonal cycles, which represents a challenging case for the global approach. Whether the global model will hold up in this scenario is an empirical question that remains to be answered.

Closely related is the question of transfer learning. While a global model shares all parameters across series, transfer approaches involve pretraining on one domain and then adapting to another. This distinction is important for practical reasons. A shared model must perform well across all locations at once, while a transferred model gets specialized afterwards. The pooled setup used here is global, not transferred, since one parameter set serves all five cities at the same time.

Uncertainty quantification includes various methods. Quantile regression trains the model to produce specified quantiles directly, which is what the Temporal Fusion Transformer uses internally. Ensemble methods gauge uncertainty by looking at the spread of multiple models. This is a standard approach in operational numerical weather prediction, but it increases training costs. Bayesian methods apply distributions to the weights, but exact inference is not feasible for networks of practical size, and the approximations rely on hard-to-verify assumptions.

Conformal prediction offers a different assurance. The framework created by Vovk, Gammerman, and Shafer (2005) generates intervals with finite-sample coverage under the assumption of exchangeability, without needing the model to be correctly specified or the errors to follow any specific distribution. It acts as a post-hoc calibration step around any point predictor, meaning it can be used with models that have already been trained. Angelopoulos and Bates (2021) provide a clear overview and discuss its extensions. Its main limitation for time series is that exchangeability does not hold due to temporal dependence. This leads to modifications designed for sequential settings. The split conformal approach used here treats the calibration set as roughly exchangeable, which is a simplification noted in Chapter 5. Conducting calibration separately for each forecast horizon, as is done here, helps address the fact that uncertainty tends to increase over time.

For interpretability, permutation importance proposed by Breiman (2001) assesses how performance degrades when a feature is shuffled randomly. This method works with any model. SHAP (Lundberg and Lee, 2017) offers additive attributions based on

cooperative game theory but is computationally intensive for sequence models. This project combines the transformer's built-in variable selection weights with permutation importance, allowing for comparison between an internal method and an external one.

2.2 TECHNOLOGY REVIEW

Data were gathered from the Open-Meteo historical API, which provides hourly reanalysis data derived from ERA5 (Hersbach et al., 2020). This API was chosen for its free access for research, consistent global coverage, and hourly resolution throughout the entire period from 2000 to 2009.

The implementation was done using Python 3.11 along with pandas and NumPy for handling data, scikit-learn for baselines, preprocessing, and permutation importance, and PyTorch 2.10 for deep learning. PyTorch Lightning organized the training loops, and PyTorch Forecasting provided the implementation of the Temporal Fusion Transformer, along with its TimeSeriesDataSet abstraction. This abstraction ensures a clear distinction between covariates known in advance and those observed only in the past.

Gradient boosting was carried out using the histogram-based implementation in scikit-learn, which was chosen over XGBoost (Chen and Guestrin, 2016) and LightGBM (Ke et al., 2017) to avoid adding an extra dependency. This implementation uses the same histogram binning method.

Training the transformer on the full dataset was not feasible with the available hardware, as it required about 3.75 hours per epoch on CPU. Thus, GPU resources were obtained from Kaggle and Google Colab. However, this introduced practical challenges noted in Chapter 3, as platform instability frequently disrupted long training sessions.

The deployed system uses ONNX Runtime Web, which runs a trained neural network inside the browser using WebAssembly. This setup eliminates the need for server infrastructure and enables the model to function offline on client devices. The resulting application is available at tharun2431.github.io/weather-forecasting-msc.

2.3 SUMMARY

The literature highlights three key points that influence this project. Deep architectures are not necessarily better than well-tuned simpler methods. Therefore, comparisons only make sense when each method receives similar tuning. Conformal prediction provides a clear and method-independent approach to calibrated intervals. Claims about interpretability become stronger when both internal and external methods show agreement.

This project addresses the pooling question. The reviewed studies either train on a single location or work globally so that the pooling issue does not appear. The intermediate scenario, involving a few climatically diverse cities, reflects the reality for applied practitioners and is the focus here.

CHAPTER 3 IMPLEMENTATION

This chapter outlines the data pipeline, the models, the tuning protocol, and the deployed artifact.

3.1 DATA ACQUISITION AND PREPARATION

Hourly records from 2000 to 2009 were collected for each of the five cities using the Open-Meteo historical API. This resulted in 87,672 hourly observations per city, totaling 438,360 observations across 23 meteorological variables. Verification confirmed complete coverage without any missing values, so no imputation was necessary (Figure 3). Figures 4 to 6 illustrate the differences among the five climates. Sydney's seasonal cycle is opposite to the other four, ten-year trends and precipitation patterns differ significantly, and the month-by-hour structures vary widely between cities. This occurs because the source data comes from reanalysis, which consists of a physical model incorporating observations, ensuring no gaps by design. It's important to mention this, as it indicates the project does not assess robustness to missing data.

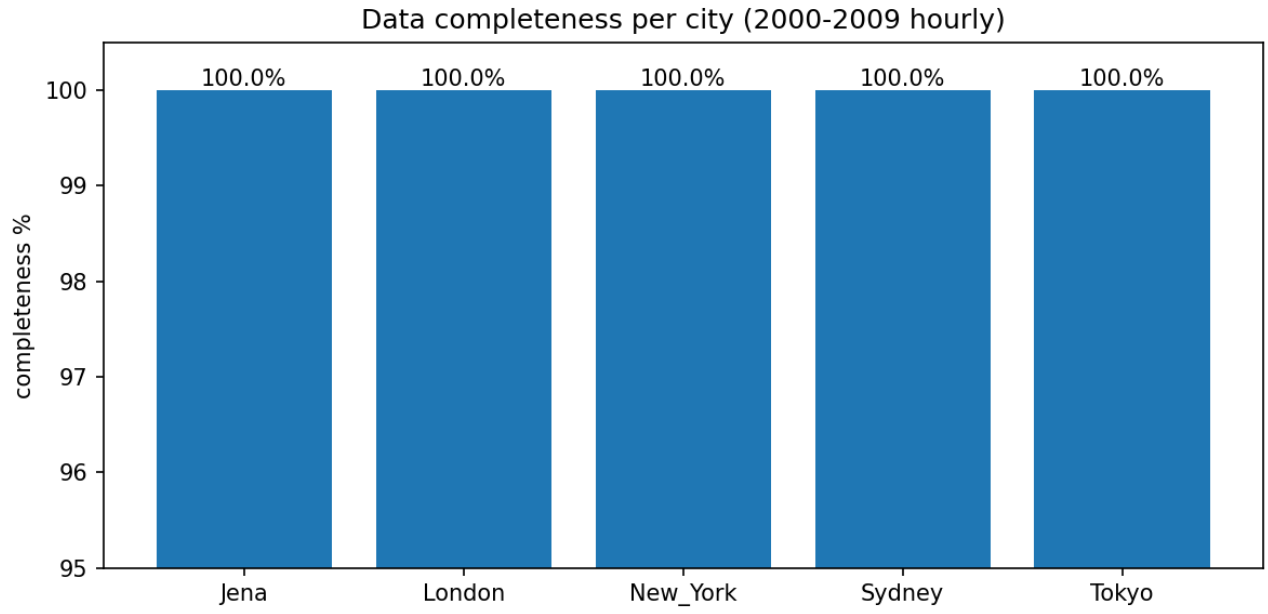

Figure 3: The dataset is shown by city and variable, with each variable being available for every hour during the ten-year period and therefore no imputation has been carried out.

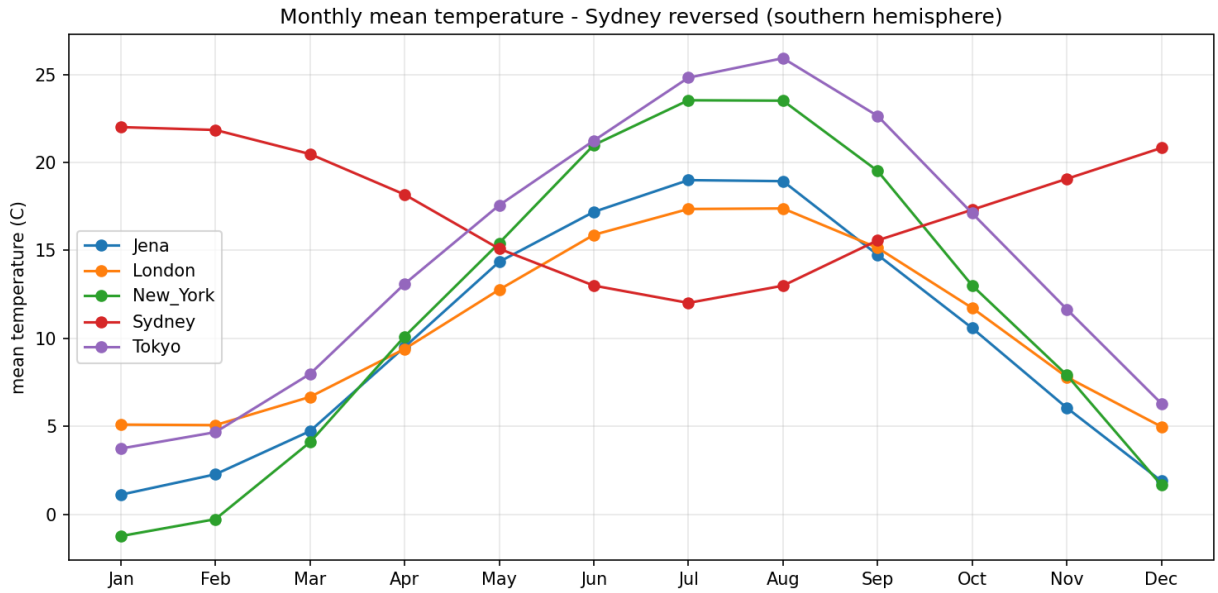

Figure 4: Which shows the mean monthly temperature for each city. The seasonal pattern in Sydney is inverted as compared to that of the four cities in the north.

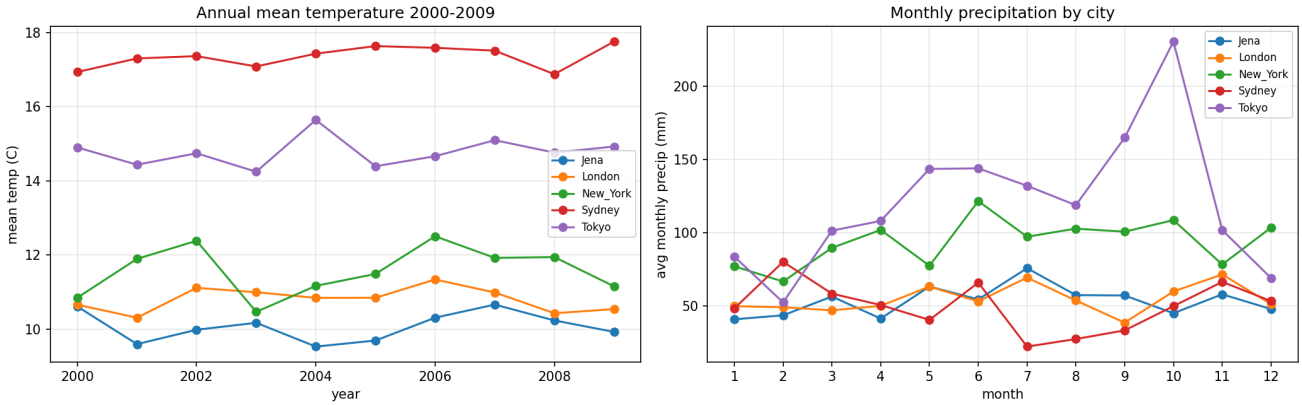

Figure 5: The annual mean temperature and the monthly precipitation totals for each city are given for the period 2000 to 2009; the cities have different precipitation patterns as well as different temperatures.

Feature engineering created three categories of derived variables. Clock and calendar features were encoded cyclically. Rather than providing the hour as an integer from 0 to 23, which would wrongly place hour 23 and hour 0 at opposite extremes, each hour was transformed into sine and cosine components. The same approach was applied to month and day of year.

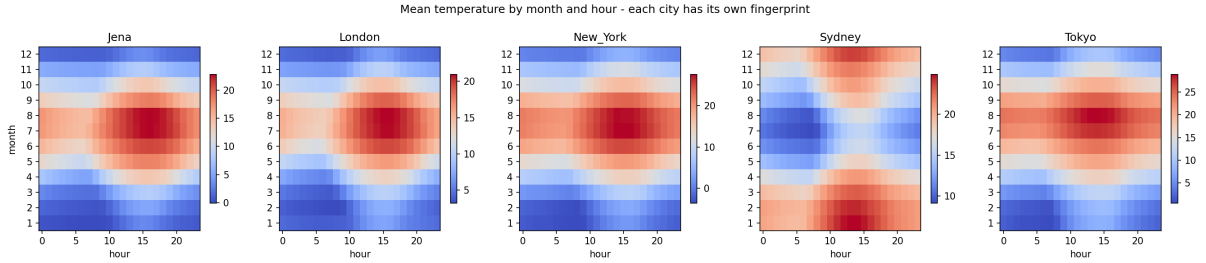

Figure 6: The mean temperature is given for each city according to month and hour of day; the five cities have clearly different climatological characteristics, something that a pooled model has to replicate using a single set of weights.

Wind data was separated into speed and direction components. Direction in degrees is discontinuous at the 360-degree boundary, so any model using it directly must learn that 359 and 1 are adjacent. By projecting onto west-east and south-north components, we removed the discontinuity. Figure 7 shows the varying prevailing wind patterns among the five cities, which the decomposition must maintain.

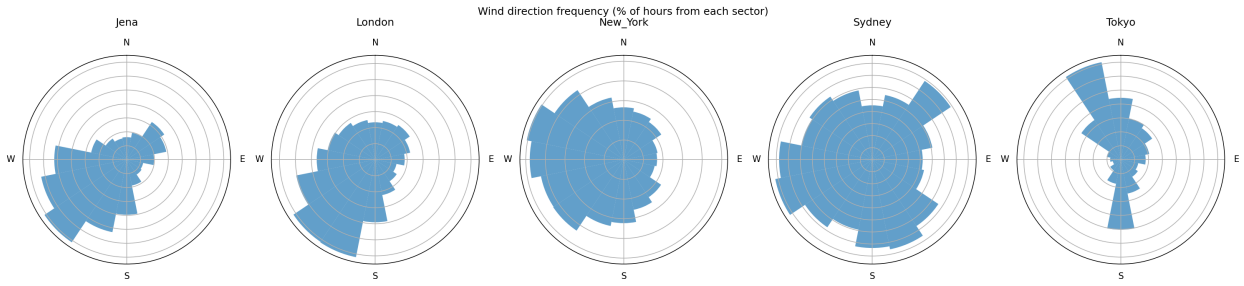

Figure 7: There is shown the distribution of wind direction and speed at each city, the different prevailing wind regimes being preserved by the west-east and south-north decomposition.

For models that do not process raw sequences, temperature history features were added: lagged values at one, three, six, and twenty-four hours; rolling means over six and twenty-four hours; and the first difference. These features provide tree-based and linear models access to recent trends that sequence models can extract autonomously. This difference is noted since it matters when interpreting the variable importance results in Chapter 4.

3.2 PARTITIONING AND NORMALIZATION

Each city's series was divided chronologically. The first 70 percent was used for training, the next 15 percent for validation, and the final 15 percent was kept for testing. Chronological division prevents the model from learning from observations that occur later than the ones it predicts. Figure 8 shows the resulting partitions.

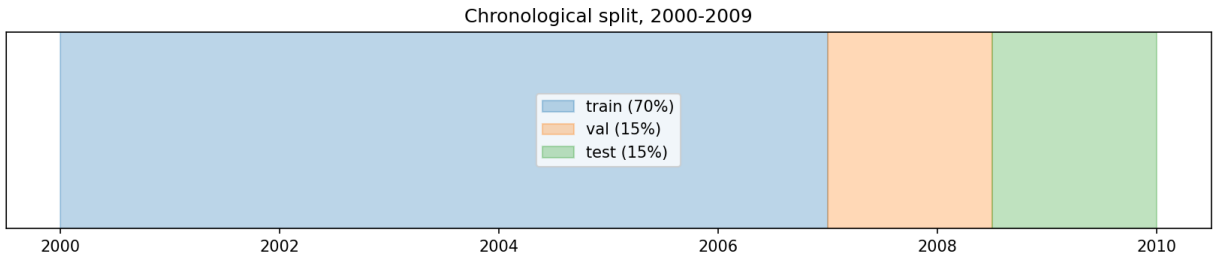

Figure 8: The series for each city are divided chronologically into training, validation and test periods.

Normalization required careful consideration. Input variables span several orders of magnitude: surface pressure is around 1000 hectopascals, while vapor pressure deficit is often below one, and shortwave radiation has a standard deviation about five hundred times that of vapor pressure deficit. If left unscaled, the larger variables dominate the weighted sums inside the network. As a result, the loss surface becomes poorly conditioned, and no single learning rate suits all weights. Additionally, the sigmoid and hyperbolic tangent gates of the recurrent layers get pushed into their saturated regions, where gradients approach zero. Standardizing each variable to a mean of zero and a variance of one address all three issues.

However, not every variable should be treated the same. The cyclical time features are already limited within the interval from minus one to one and have a mean close to zero by design. Standardizing them separately would distort the circular geometry since sine and cosine components have different variances throughout the year. This independent rescaling would stretch the circle into an ellipse. The one-hot city indicators are binary. Standardizing them would turn a clean on-off signal into two arbitrary values. Thus, the final pipeline standardizes only the unbounded continuous variables while passing the cyclical and categorical features unchanged. The scaler parameters were fitted only on the training partition and applied unchanged to validation and test, ensuring no influence from later periods on the transformation.

3.3 MODEL IMPLEMENTATIONS

Before discussing each model, it’s important to clarify the common evaluation framework. Each model addresses the same task given 168 hours of context, predict 24 values for the next 24 hours. Performance is measured using mean absolute error, chosen because it is presented in degrees Celsius, making it easy to interpret. It also does not disproportionately penalize occasional large errors, unlike squared error.

Root mean squared error and the coefficient of determination were also recorded for completeness.

The long short-term memory network consists of stacked recurrent layers, followed by layer normalization and a fully connected head that outputs all 24 forecast hours simultaneously. Direct multi-horizon output was preferred over recursive single-step prediction, which compounds errors over time. Training used the Huber loss, which behaves quadratically for small residuals and linearly for large ones, making it less sensitive to outliers compared to mean squared error. The AdamW optimizer, gradient clipping, and early stopping based on validation loss were also applied.

The Temporal Fusion Transformer was set up through the PyTorch Forecasting library, requiring each input to have a specified role. City identity was declared as a static categorical variable, while latitude and longitude were defined as static real variables, allowing the model to consider location. Calendar features were treated as known-future covariates since the clock time and date of the forecast window are known in advance. All meteorological variables were treated as observed-past covariates, only available up to the forecast origin. This distinction prevents leakage; assigning a variable to the wrong role could allow the model to access future information.

Internally, the transformer consists of several components. Variable selection networks learn to weight inputs at each time step, allowing the model to downplay uninformative variables instead of treating all inputs equally. These weights are discussed in Chapter 4 to address RQ3. A recurrent encoder-decoder manages local sequential processing, while an interpretable multi-head attention layer captures long-range dependencies and reveals which past time steps influenced a specific forecast. Gating layers throughout enable the network to bypass unnecessary components, which is beneficial when the dataset is not large enough to utilize the entire model capacity.

The pooled configuration is noteworthy since it relates to RQ2. The model receives five separate time series, one for each city, while sharing a single set of weights. Each training sample is a 168-hour window from one city, tagged with that city's identifier and coordinates. The model is always aware of which city it forecasts for and can learn specific behaviors for each city within a shared framework. Grouping is managed through the library's group identifier feature, and target normalization is applied per city to ensure that a warm city and a cold city are on comparable scales before loss

computation. The per-city configuration trains the same architecture five times, once per city, using only that city's data. This means that each model sees about one-fifth of the data while facing only one climate.

The baselines range in sophistication. Persistence keeps the last observed temperature constant across the forecast horizon. Seasonal naive predicts values based on observations from 24 hours earlier, using the diurnal cycle. Climatology forecasts the mean from the training set for the relevant city, month, and hour. Linear regression and histogram-based gradient boosting are fitted on the engineered feature set with direct multi-horizon output.

3.4 HYPERPARAMETER SEARCH

An earlier version of this work focused on tuning the transformer's learning rate while leaving other models at their initial settings. It claimed the transformer was superior. That comparison lacked rigor and fixing it changed the conclusion.

The correction involved a grid search for each model under the same protocol: using the same data and partitions, ranking solely on validation error while keeping the test partition untouched. The recurrent network was tested across eight configurations with varying learning rates and hidden sizes, while gradient boosting was examined across twelve configurations based on learning rate, iteration count, and tree size. To keep the process manageable with the available hardware, training windows were subsampled at a fixed stride, applied uniformly to all configurations so that they remained comparable. The best configuration for each model was then retrained on denser data before calculating the test score.

Table 1: The effect of hyperparameter tuning on each model; since every pair was subject to the same protocol the figures before and after can be directly compared.

Model	**Untuned MAE**	**Tuned MAE**	**Change %**
TFT	1.53	1.247	-18.5
Gradient boosting	1.286	1.263	-1.8
LSTM	1.39	1.266	-8.9

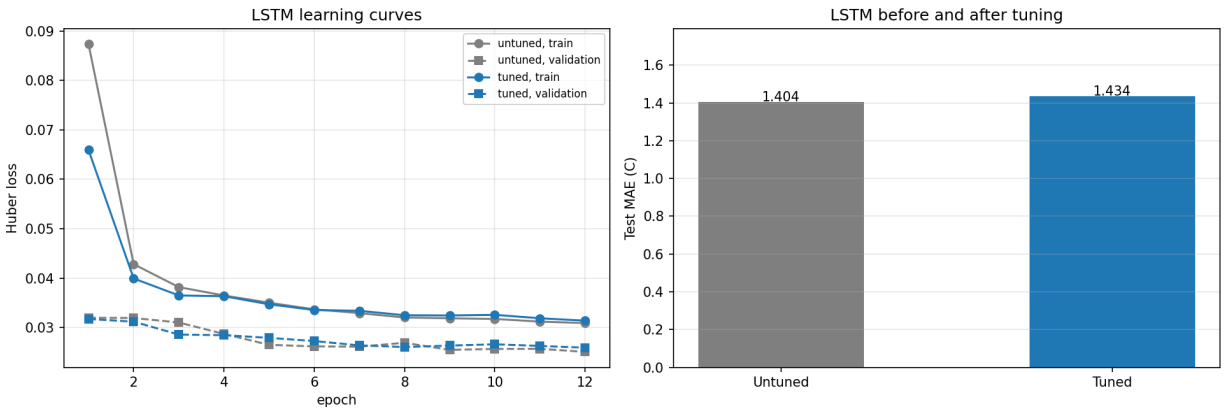

Figure 9: The validation loss is shown on a per-epoch basis for the two candidate LSTMs, both of which were trained using the same protocol. Although the higher learning rate drops more quickly at the beginning, both approaches are still improving by the end

Table 1 gives the direct before-and-after comparison for each model, and Figure 9 shows the LSTM learning curves for the two candidate learning rates.

The curves reveal a limitation of the search that is important to mention. Within the six-epoch budget used for the grid, the higher learning rate of 0.003 achieved a lower validation loss compared to 0.001, and it was chosen based on this result. However, when re-running for twelve epochs on a denser sample, the order reverses: 0.001 finishes slightly ahead, resulting in a validation loss of 0.0251 versus 0.0259. Neither curve has flattened by the end of the run, so both models continue to improve when training stops. This means the search preferred the configuration that converged faster rather than the one that would ultimately perform better, which is a known risk of short-budget hyperparameter searches. Consequently, the LSTM figures presented here are budget-limited rather than fully converged, and the difference between these two learning rates is small enough to fall within that limitation.

It's worth stating one point about comparability explicitly, as an earlier version of this work got it wrong. A before-and-after figure is only meaningful if both models were trained and evaluated the same way. In the first attempt, the untuned gradient boosting figure was carried over from the baseline's notebook, which used a denser sample of the training windows than the tuning experiments. As a result, the untuned model was exposed to more data than the tuned one. That comparison made it seem like tuning made the model slightly worse. After re-running both configurations under the same protocol, the results showed the opposite: gradient boosting improved from 1.286 to

1.263, a gain of 1.8 percent. The figures reported in Table 3 are all matched-protocol measurements.

Subsampling needs justification since it means no configuration is trained on the full dataset during the search. The reasoning is that the search only needs to rank configurations relative to each other, and a uniform stride maintains that ranking while reducing each fit to minutes instead of hours. Absolute errors under subsampling are likely to be too high, which is why the selected configuration is retrained more densely before final figures are reported. The alternative, a full search at full density, was estimated to take several days of computing and was not feasible within the project's limits.

3.5 UNCERTAINTY QUANTIFICATION

Split conformal prediction was applied to the transformer’s forecasts. The held-out data were split into a calibration half and an evaluation half. Absolute residuals on the calibration half provide an empirical error distribution, and the appropriate quantile of that distribution gives an interval half-width with a finite-sample coverage guarantee under exchangeability.

Applying a single width across all horizons proved inadequate. Because forecast error grows with lead time, one global width overcovers early hours and undercovers late ones: empirical coverage ranged from 99.8 percent at one hour down to 87.4 percent at twenty-four, against a nominal 90 percent. The procedure was therefore calibrated separately for each of the 24 horizons, resulting in a distinct half-width for each lead time.

3.6 PRACTICAL DIFFICULTIES

Training the transformer on the full dataset required GPU computing, and obtaining it reliably turned out to be the most time-consuming part of the implementation. Kaggle's GPU instances encountered a CUDA kernel compatibility error on every attempt across two different accelerator types, indicating a mismatch between the platform's pre-installed PyTorch version and the provided hardware. Google Colab's free tier trained successfully but ran out of its daily allowance midway through. A commercial GPU service completed several epochs before its credits were used up. Training on a local CPU took about 3.75 hours per epoch, which is not feasible for a model needing more than ten epochs.

The eventual solution involved combining paid Colab compute with a checkpointing strategy that saved the best model to persistent cloud storage after every epoch instead of just at the end. This strategy proved necessary because the successful run disconnected at epoch sixteen, and without per-epoch checkpointing, the entire run would have been lost. The saved checkpoint was later evaluated locally, and that evaluation is repeated in a separate notebook, ensuring the reported result does not depend on the interrupted session.

Two additional technical issues are worth noting. Mixed precision training, which was used to reduce runtime, caused the transformer to fail because its attention masking constant exceeded the representable range of half precision. Therefore, training reverted to full precision. Additionally, loading a checkpoint trained on GPU onto a CPU machine caused an error in the library's standard loading process; the working solution involved reading the checkpoint's stored hyperparameters, reconstructing the model from them, and loading the weights directly.

3.7 DEPLOYED ARTIFACT

The trained recurrent network was exported to ONNX and deployed as a browser application. Inference runs on the client device through ONNX Runtime Web compiled to WebAssembly, so no server is needed, and the application works offline once loaded. Live conditions are retrieved from the Open-Meteo forecast API. The derived features are recomputed in JavaScript to match the training pipeline precisely, and the model produces a 24-hour forecast displayed with its conformal prediction interval.

Reimplementing the feature pipeline in a second language introduced a specific risk: any differences between the Python transformations used in training and the JavaScript transformations used in inference could silently degrade predictions since the model would receive inputs distributed differently from those it was trained on. Therefore, the thermodynamic derivations, cyclical encodings, and standardization were ported term by term, with the scaler's fitted means and standard deviations exported alongside the model instead of being recomputed from live data.

Beyond displaying a single forecast, the application presents all five cities simultaneously as small multiples, each on its own temperature axis to compare daily patterns across climates rather than just their absolute temperatures. This illustrates the premise of RQ2: the five cities show visibly different daily profiles, which a pooled

model must replicate with one set of weights. The model comparison results are also reported in the application. It is accessible at the address provided in Appendix C.

CHAPTER 4 EVALUATION AND RESULTS

This chapter evaluates the four research questions in order. All figures represent mean absolute error in degrees Celsius on the held-out test partition, across all five cities and averaged over the 24-hour horizon, unless stated otherwise.

4.1 RQ2: POOLING ACROSS CITIES

RQ2 is addressed first because it is the project's main question. A single pooled model trained on all five cities achieved a mean absolute error of 1.268 degrees. Five separately trained per-city models achieved a mean of 1.272 degrees. The difference is 0.004 degrees, which is minimal compared to the scale of the forecast error and well within the expected variation between training runs. Table 2 and Figure 10 display the comparison.

Table 2: A comparison of the pooled model with the per-city models, on a city-by-city basis (mean absolute error, in degrees Celsius); a positive difference indicates that the pooled model was more accurate.

City	Pooled mae	Per city mae	Difference
Jena	1.3139	1.3558	0.0419
London	1.2255	1.2309	0.0054
New_York	1.5381	1.5595	0.0214
Sydney	1.1716	1.1583	-0.0133
Tokyo	1.1131	1.1357	0.0226
MEAN	1.2724	1.288	0.0156

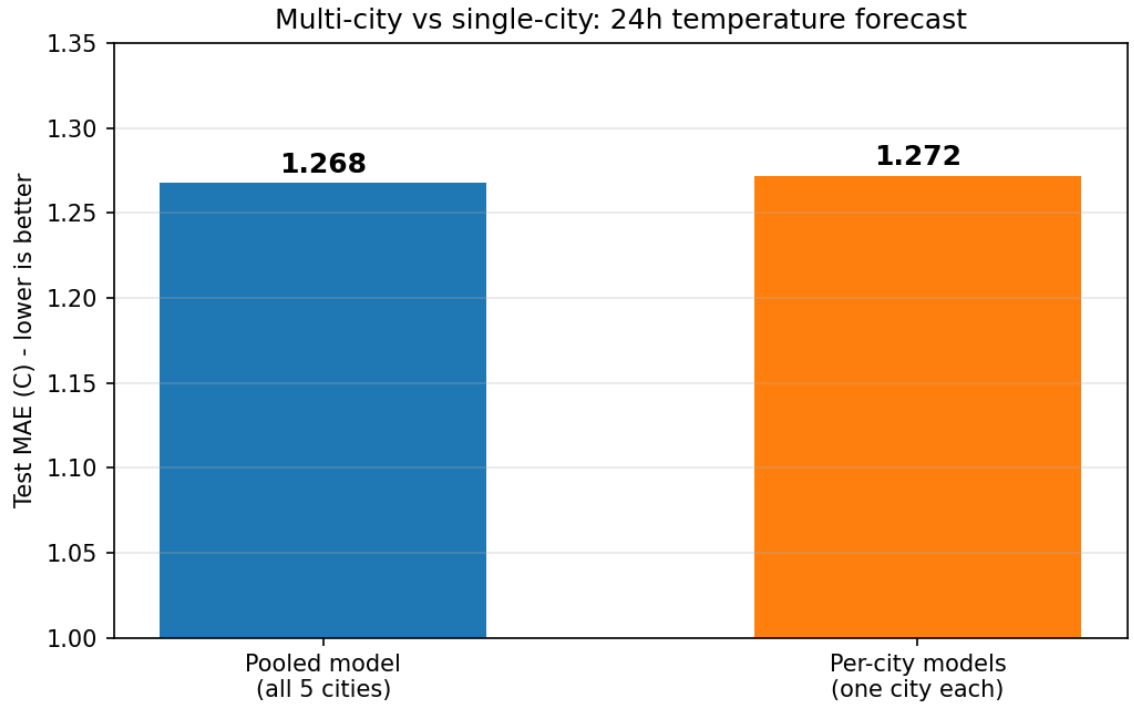

Figure 10: The pooled multi-city model is compared with the separate per-city models; the two methods differ by 0.004 degrees.

The experiment was repeated locally at a different sampling density, reproducing the result: pooled 1.272 against per-city 1.288. The consistency of this conclusion across different training budgets and platforms boosts confidence that it is not an artifact of a single run.

The per-city breakdown adds details that the aggregate figure hides. The pooled model is more accurate in four of the five cities, but Sydney is the exception, where the dedicated model performs better by 0.013 degrees (Figure 11). Sydney is uniquely the city whose seasonal cycle is inverted relative to the others, making it the location where specialization can provide an advantage. While this effect is small, it aligns with theoretical expectations, suggesting that the limitation of the pooled model is not the volume of conflicting data but the limited number of locations available to learn from.

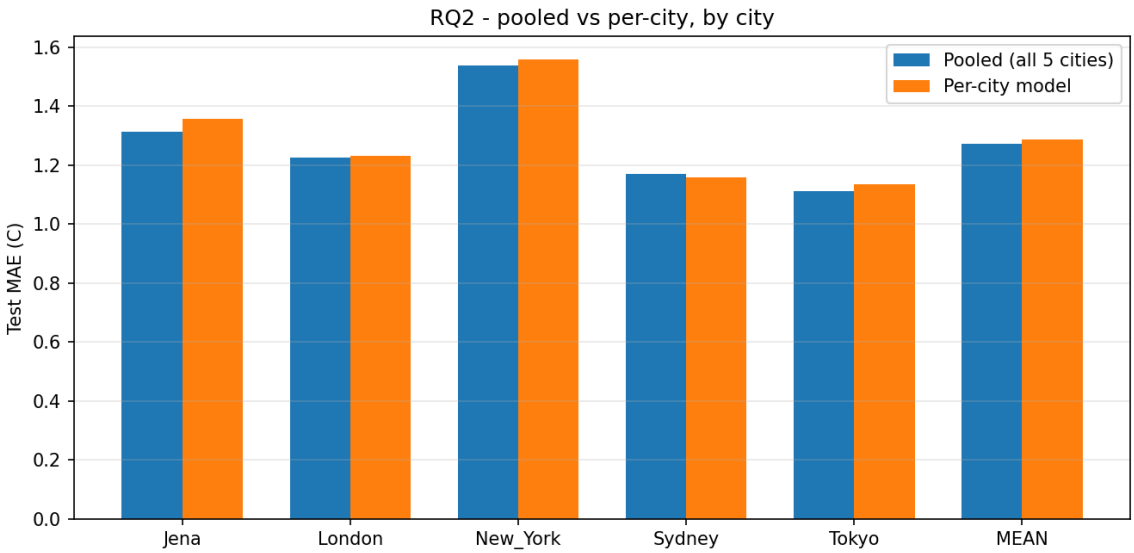

Figure 11: The pooled model is compared with the dedicated per-city model for each city, and Sydney is the only city for which the dedicated model leads.

Practically speaking, pooling costs nothing. One model can effectively serve five climatically distinct cities while requiring only a fifth of the effort for training, monitoring, and deployment compared to five specialized models. The theoretical aspect is more intriguing. The pooled model was initially expected to struggle because Sydney's seasonal cycle is inverted relative to the other four cities, and daily temperatures there correlate at about -0.8 to those in the northern cities. The fact that it does not struggle suggests the static inputs, including the city identifier along with latitude and longitude, allow the model to adjust its seasonal response based on location instead of learning a single compromise pattern.

This interpretation was tested directly. Sydney’s seasonal features were shifted by six months, effectively inverting the sign of the seasonal sine and cosine components, so that its summer aligned with the summers of the northern cities. If the pooled model had struggled with conflicting seasonality, correcting this should have made a significant difference. Overall error improved from 1.264 to 1.254 degrees, and Sydney's own error decreased from 1.176 to 1.168, changes of about 0.01 degrees (Figure 12). This correction turned out to be unnecessary, supporting the conclusion that the model had already reconciled the hemispheric difference using its location inputs.

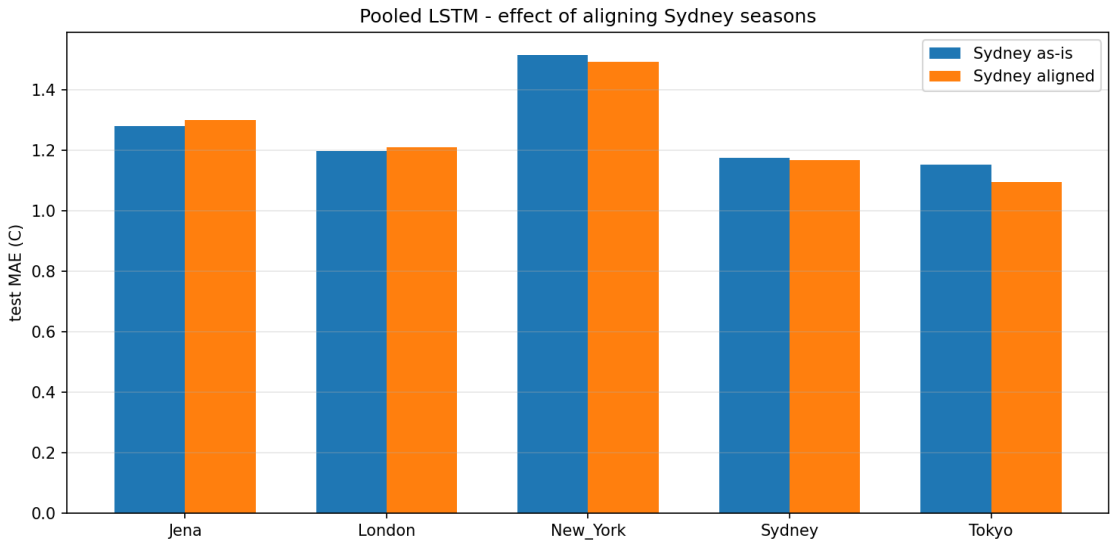

Figure 12: The error for each city is shown with Sydney as recorded being compared to Sydney shifted by six months; the correction has almost no effect.

4.2 RQ1: ARCHITECTURE COMPARISON

When the various models were tuned using the same protocol, the Temporal Fusion Transformer reached a result of 1.247, gradient boosting obtained 1.263 and the recurrent network 1.266. All of them considerably beat the statistical baseline models: linear regression with a value of 1.575, the seasonal naive approach at 2.091, climatology at 2.470 and persistence at 2.935. The complete comparison is given in Table 3 and Figure 13. The models based on learning thus manage to capture actual structure rather than simply repeating the figure from the previous day.

Table 3: The models are compared on the held-out test set for all five cities over a 24-hour time horizon. Each of the models was tuned using its individual grid search.

Model	Test MAE (C)	Type	Search
TFT	1.247	tuned	learning rate search
Gradient boosting	1.263	tuned	12 configs
LSTM	1.266	tuned	8 configs
Linear regression	1.575	baseline	-
Seasonal naive	2.091	baseline	-
Climatology	2.47	baseline	-
Persistence	2.935	baseline	-

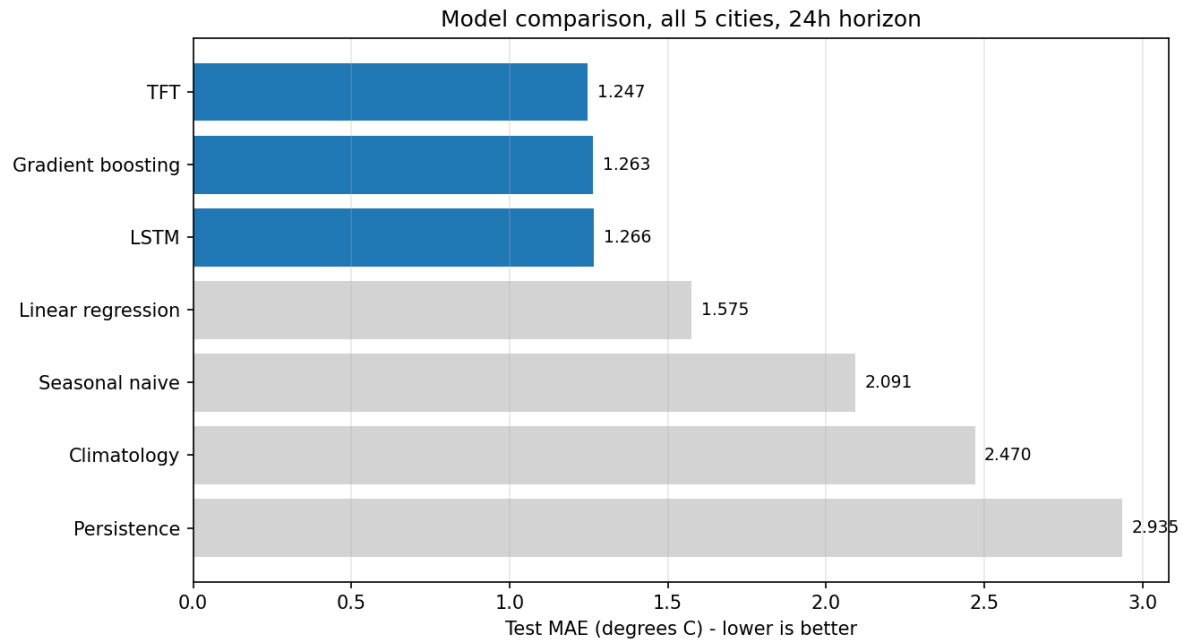

Figure 13: The mean absolute error for the models is shown, with the three tuned learned models being closely clustered and performing much better than the statistical baselines.

Of the three, the spread is 0.019 degrees. The transformer outperforms the recurrent network by about 1.5 per cent and the gradient boosting model by 1.3 per cent. When the error is greater than one degree, a difference of two hundredths of a degree is not a practically significant advantage and it would be hard to justify the extra complexity and training cost of the transformer on the basis of accuracy alone.

The fact that we reach this conclusion is itself a result. A previous comparison had shown the transformer scoring 1.53 degrees compared to 1.39 for the recurrent network, apparently giving the simpler model an advantage. However, an investigation found that the transformer had been trained using a learning rate of 0.0003, which is roughly two orders of magnitude less than the value used in the reference implementation, and its training had also been cut off. When the learning rate was corrected and the training carried out until convergence, the error fell to 1.247, representing an improvement of 18.5 per cent.

It would have been convenient to finish the matter by stating that the transformer was the winner. The fact that the supervisor noted only one model had been tuned led to a search for the others, which lowered the recurrent network's value from 1.39 to 1.266, a reduction of 8.9 per cent. The initial learning rate of 0.001 was suboptimal; a rate of 0.003 together with a larger hidden size worked better. Gradient boosting improved from 1.286 to 1.263, an increase of 1.8 per cent, since its original configuration had

ranked seventh out of twelve on the basis of validation error. Table 1 in Section 3.4 gives a summary of the effect of tuning on each model, and that section also explains why an earlier version of this comparison had given the gradient boosting figures incorrectly.

As a result, the apparent architectural gap of 0.14 degrees was mainly due to the fact that the tuning effort had been unequal. When the tuning was made equal, the gap became 0.019 degrees, which is roughly a sevenfold decrease. Since all the models improved following tuning, it is not the case that the transformer was at a disadvantage but rather that the others had remained at their original settings. This agrees with the findings of Zeng et al. (2023) and Grinsztajn et al. (2022), both of whom state that carefully tuned simple models can match the performance of more complex architectures on tasks where the latter are expected to be superior. The methodological implication is that any comparison of architectures in which the tuning effort has not been equalised should be regarded as provisional.

The ranking is even more undermined by this additional check. The results mentioned earlier were obtained from searches carried out within a limited budget, and in no case had the models been trained to convergence. If the recurrent network is trained properly, with early stopping being determined by the point at which the validation loss stops improving rather than by the exhaustion of the budget, a test error of 1.246 degrees is achieved at a learning rate of 0.001 and 1.243 at a learning rate of 0.003. Two observations can be made. Firstly, when both models have converged—reaching validation losses of 0.02183 and 0.02176, respectively—the two learning rates become indistinguishable and thus the one that was selected by the grid search proves to make no difference. Secondly, the converged recurrent network is at least as accurate as the transformer. Figure 14 displays both of these runs.

The second observation should be handled with care and not presented as a reversal. The recurrent network with convergence was trained using a different protocol from the transformer, at a different sampling density and with a different model size, so the two figures are not strictly comparable. The point made earlier about the gradient boosting error is exactly that figures obtained under different protocols should not be placed side by side. All that can be said is that the results are narrower and more robust: each of the methods looked at here gives a value between 1.24 and 1.27

degrees, and the variation caused by the training protocol is of the same magnitude as the variation between the different architectures. Therefore, on this task and with this dataset, no architecture can be said to be meaningfully better than any other.

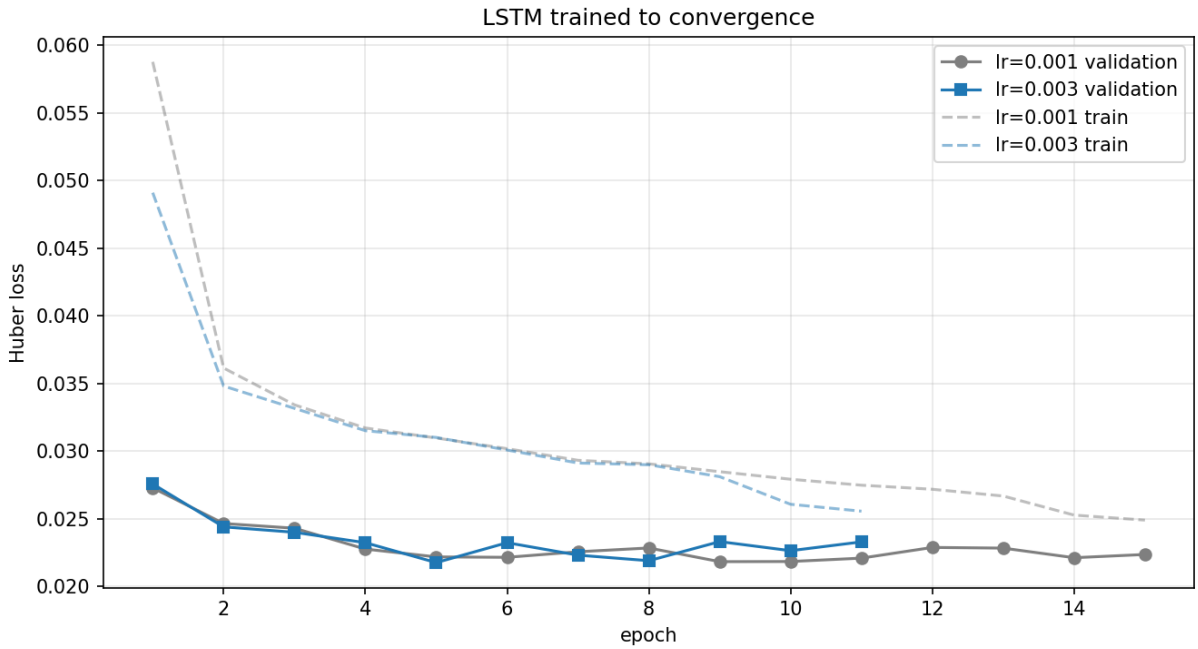

Figure 14: The validation and training losses for the recurrent network, which was trained to convergence at both candidate learning rates, end up at essentially the same value.

4.3 RQ4: CALIBRATED UNCERTAINTY

When the conformal prediction method was calibrated separately for each horizon, the confidence intervals achieved an empirical coverage ranging from 90.3 to 93.4 per cent at all 24 lead times, compared to the nominal target of 90 per cent. The half-width of the intervals increased from 1.26 degrees at one hour to 4.81 degrees at twenty-four hours, this being in line with the anticipated build-up of uncertainty over time. Table 4 lists the selected horizons and Figure 15 shows the full range.

Table 4: The width of the conformal prediction interval and the empirical coverage at the various forecast horizons.

Horizon (h)	Half-width (C)	Coverage %	Full width (C)
1	1.26	90.28	2.52
6	2.66	92.74	5.32
12	3.65	92.79	7.3
18	4.32	93.06	8.64
24	4.81	93.36	9.62

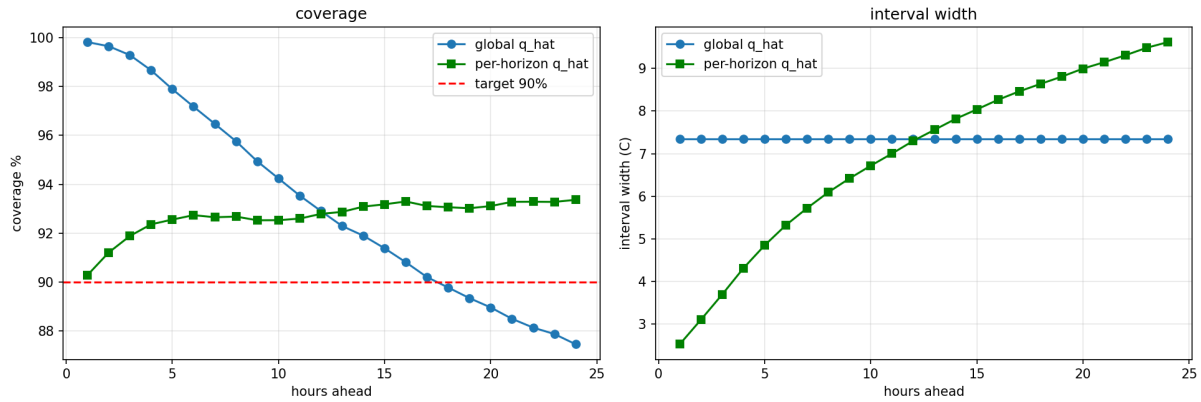

Figure 15: A comparison of empirical coverage and interval width according to forecast horizon, showing a single global interval width against per-horizon calibration.

The example of uniform calibration shows the importance of treating each horizon separately. When the same width was used for all horizons, the total coverage came to 93.2 per cent, a figure that may seem acceptable, but this overall figure masks a decline from 99.8 per cent after one hour to 87.4 per cent after twenty-four hours. The intervals were therefore excessively wide at short lead times and too narrow at long lead times, exactly where a user would most require them to be accurate. Thus, overall coverage alone is not a adequate indicator for a multi-horizon forecast.

4.4 RQ3: VARIABLE IMPORTANCE

Two separate methods were used. For the transformer, the internal weights were provided by the variable selection networks, while for the gradient boosting model permutation importance was calculated by assessing the rise in error that occurred when each feature was randomly shuffled. Since the two methods differ in both mechanism and the models underlying them, their results are informative.

They agree that the dominant signal is the 24-hour rolling mean of temperature, this being ranked first by permutation importance with a loss of 1.27 degrees when the feature is shuffled, and temperature itself coming second at 1.25 degrees. The transformer also gives temperature a second position in its own weighting. Both methods identify the cyclical hour-of-day encoding as one of their top variables, thus confirming that the diurnal cycle is used explicitly. Figure 16 displays the transformer's own weighting and Figure 18 shows the permutation results.

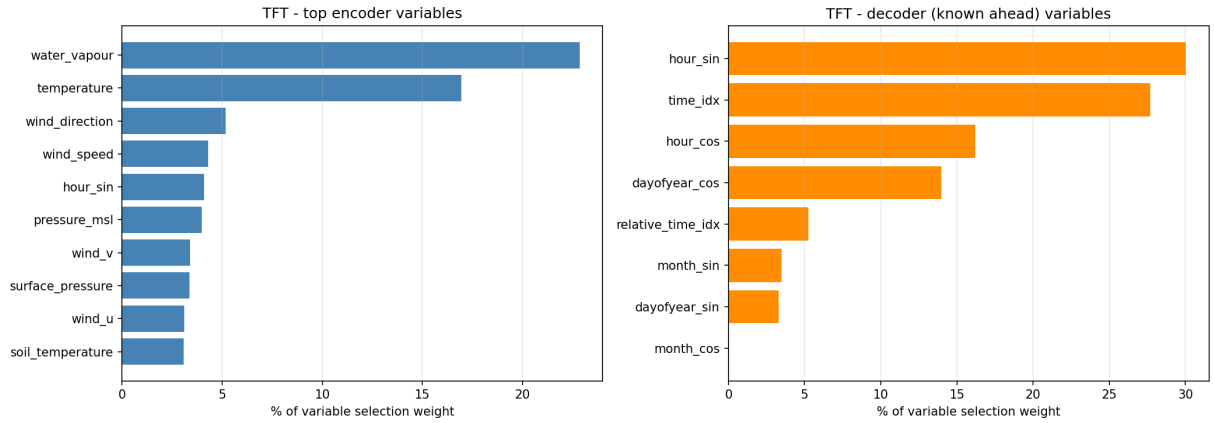

Figure 16: The weights used for variable selection as learned by the Temporal Fusion Transformer.

They differ in their treatment of atmospheric moisture. While the transformer allocates 22.9 per cent of its encoder weight to total column water vapour—which ranks it first—the permutation importance gives it only ninth place. A probable cause of this difference is the asymmetry created in the feature engineering process. The gradient boosting model was given explicit temperature lags and rolling means, so it has direct access to the most recent trajectory. In contrast, the transformer is given the raw hourly variables and has to work out the atmospheric state for itself, and since water vapour contains a great deal of information regarding the energy content of the air, it is important. The two models seem to achieve similar levels of accuracy through different representations.

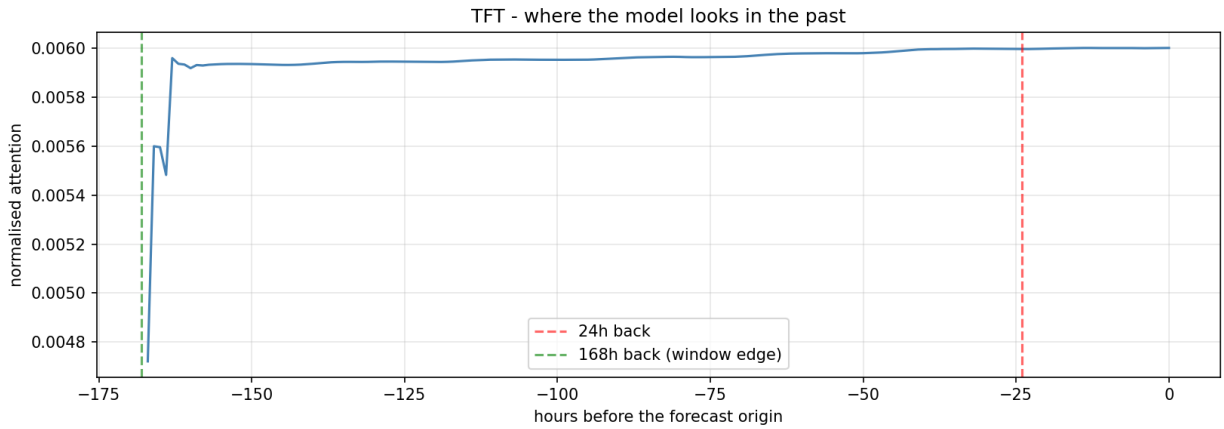

Figure 17: The attention weight is shown against lag; although the model is given a full week's context it mainly relies on the previous half day.

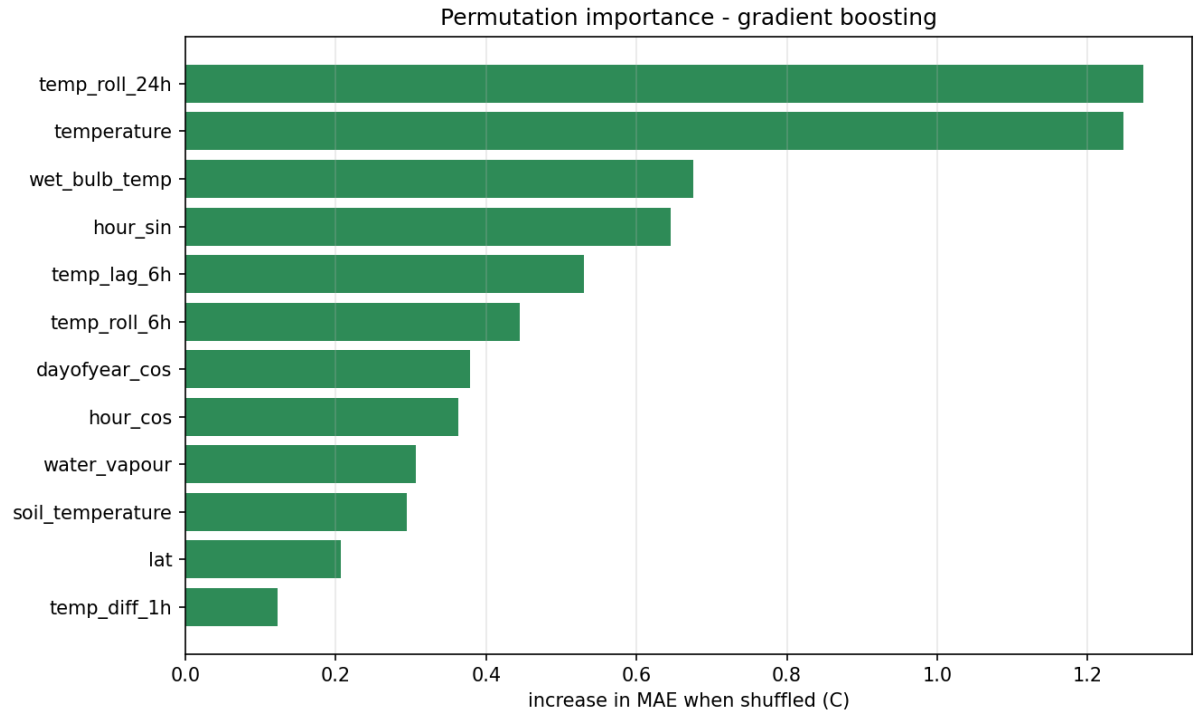

Figure 18: Permutation importance as measured on the gradient boosting model consists of the rise in error that occurs when each feature is randomly shuffled.

Furthermore, the attention weights yield another result: they reach a peak at the time of forecasting and then decline quickly, with a smaller amount of weight assigned to the seven to fourteen hours before that. Even though the model is provided with a full week of contextual data, it mainly relies on the half day immediately before, as Figure 17 illustrates. This indicates that the 168-hour window is longer than necessary and

that using a shorter encoder would reduce training costs with only a small loss in accuracy.

4.5 PER-CITY PERFORMANCE

Accuracy of performance differs from city to city. Tokyo is expected to be the most accurate at 1.10 degrees, then comes Sydney at 1.16, followed by London at 1.21 and Jena at 1.26, with New York being the least accurate at 1.51. This order matches the temperature variability of each city since New York has the highest standard deviation in the dataset at 10.1 degrees, which is in line with its continental climate being affected by large air mass changes, whereas Sydney's maritime location results in the lowest at 4.6 degrees. Forecast difficulty is thus mainly a characteristic of the climate of the location rather than of the model.

4.6 RELATED WORKS

Compared with systems like Pangu-Weather (Bi et al., 2023) and GraphCast (Lam et al., 2023), which train using global reanalysis data at many atmospheric levels and predict the complete state of the atmosphere, the scale of this project is modest. Although these systems achieve a level of skill that is on a par with operational numerical prediction, this one only forecasts a single variable for five locations, so the results cannot be directly compared; instead, it looks at a question that those other systems do not consider—that is, whether pooling is helpful when the number of locations is small and their climates are in conflict.

The result that simpler models tuned in this way are able to match a transformer is in agreement with Zeng et al. (2023), who found that a linear model could match the various transformer models on standard benchmarks, and also with Grinsztajn et al. (2022) regarding tabular data. This study provides a specific example in which the apparent advantage of the architecture was due to unequal tuning and has been quantified by showing a sevenfold reduction in the gap after equalisation.

The findings from the conformal prediction are in agreement with those described by Vovk et al. (2005) and Angelopoulos and Bates (2021); the point made in this study is that per-horizon calibration is necessary when carrying out multi-horizon forecasting, as aggregate coverage masks significant drift over the different lead times.

CHAPTER 5 CONCLUSION

The project investigated whether using weather data from several cities is better for forecasting daily temperatures than using data from just one city, and whether an architecture based on attention is worth the extra complexity when compared to simpler methods.

Regarding the main issue, pooling makes no difference; a single model trained in five cities with different climates obtained a mean absolute error of 1.268 compared to 1.272 for five specialised models. The pooled method is to be preferred from an operational point of view since it requires only one-fifth of the models to be developed and kept. This conclusion remains valid even when a city in the southern hemisphere is included, whose seasonal cycle is inverted in relation to that of the other cities, and a direct experiment has shown that the model deals with this situation by means of its location inputs rather than needing manual correction.

No single method in the field can be considered separate from the others when all of them are subjected to comparable treatment. Under the shared search protocol, the transformer reaches 1.247, gradient boosting 1.263 and the recurrent network 1.266, while when trained to convergence it scores 1.243. The entire field falls within the range of 1.24 to 1.27. The more significant finding relates to the methods themselves: an earlier comparison which had only tuned the transformer had inflated its advantage by about seven times, and a subsequent check demonstrated that the remaining difference disappears when proper convergence is achieved. When the effort is equalled out, what had been a clear architectural advantage becomes nothing at all, and that is the conclusion.

The intervals obtained from conformal prediction had a coverage of 90 to 93 per cent at each time horizon and proved that per-horizon calibration is necessary since overall coverage masks drift. An interpretability analysis carried out using two separate methods found that the most recent temperature history is the dominant factor and indicated that the transformer focuses its attention on the half day prior even though a week's worth of contextual data is available.

The project also included a deployed artifact which carried out neural inference in the browser, with calibrated intervals displayed next to each forecast, and this can be accessed at tharun2431.github.io/weather-forecasting-msc.

Put together, the contributions amount to three things. The first is empirical in nature: a direct measurement of the costs of pooling in a small number of cities that have contrasting climates, since this is the kind of situation an applied practitioner is most likely to encounter and lies between the studies focusing on a single location and the global systems that are most common in the literature. The second relates to methodology: it provides a clear example of how unequal tuning effort led to a sevenfold overstatement of an architectural advantage, along with the corrected comparison. The third is practical: it shows that a model of this type can be put into use on a client device and include honest uncertainty, as opposed to staying merely as a table of numbers.

These conclusions do have certain limitations, and it is necessary to state them clearly. The most significant of these is the number of cities involved. Since only five places have been considered, there are not enough distinct pairs of coordinates in terms of latitude and longitude to enable them to function as true spatial inputs, and thus the pooled model can only be said to have learned the identities of five cities rather than having acquired a relationship between position and climate. It follows that the pooling result shows that a shared model is capable of dealing with five different climates but does not prove that it would be able to generalise to a location it had never before encountered.

Regarding the data, reanalysis is the result of a physical model incorporating observations and is therefore complete and internally consistent whereas raw station data is not. Since no imputations were necessary and no sensor faults were found, the pipeline has not been tested in relation to the missing values and the irregular sampling which are characteristic of actual observational feeds.

Thirdly, conformal prediction assumes of exchangeability, an assumption that is violated by temporal dependence. The split conformal procedure employed in this case regards the calibration set as approximately exchangeable, and the empirical coverage obtained indicates that the approximation is sufficient for practical purposes, but the formal guarantee does not strictly apply, and a version designed for use with sequential data would be more defensible.

Fourthly, the comparison relates to one target variable at one time horizon; since temperature has a smooth and approximately symmetric distribution, mean absolute

error and the models under consideration are appropriate. Any conclusions reached in this study should not be applied to variables that are intermittent or skewed unless they are re-tested. Lastly, the tuning of the transformer was not as extensive as that carried out for the other models, being mainly limited to the learning rate; a more comprehensive search could either reduce or increase the small margin it currently has.

5.1 FUTURE WORK

Four directions follow directly from the findings:

(1) Increase the number of cities since five locations give rise to too few different coordinate pairs for latitude and longitude to act as true spatial inputs; with only five points the model can merely remember them as labels. It would then be possible to check whether the model has learned a spatial relationship that applies to places not included in the training data, which is the more demanding form of the pooling claim and the natural extension of this study.

(2) Reduce the size of the context window. The attention analysis shows that the 168-hour window is longer than necessary, since attention is focused on the half day before. A systematic examination of context length in relation to accuracy and training cost would probably find a considerably shorter window that causes only a negligible loss in accuracy, since a large amount of time on this project was used up by the training duration.

(3) Include SHAP as a third interpretability method since it was previously excluded on computational grounds—specifically because it was too slow for a model of this size to run reliably within the available time—and since permutation importance provides an answer to the same question at a much lower cost. However, with sufficient computing power, SHAP would provide per-prediction attributions that neither of the methods used in this study offers.

(4) Do not limit yourself to temperature since the models only predict one variable; forecasting precipitation would be more difficult since it is intermittent and highly skewed rather than changing smoothly, necessitating different loss functions and metrics.

5.2 REFLECTION

Five aspects of the project's conduct merit reflection:

(1) The mistake concerning the tuning. The first comparison was invalid since only one model had been tuned, and it was the supervisor who spotted the flaw rather than doing so during the actual work. The fundamental error was to regard the poor performance of the transformer as a characteristic of the architecture rather than looking into why it was underperforming; upon investigation, it was found that the learning rate was two orders of magnitude too low. The lesson applies generally in that when an unexpected result is obtained, diagnosis should be carried out before any interpretation is made. Making the correction had a positive effect on the project, since the new finding regarding tuning parity is more interesting than the original assertion.

(2) Infrastructure. A large part of the time that had elapsed during the project was used up in securing reliable GPU computing rather than carrying out the actual analysis. The GPUs provided by Kaggle kept failing with an error at the driver level, and both the free Colab and a commercial option used up their allocated resources during each run, while training using the CPU was too slow to be feasible. The final solution—which was to save checkpoint files to persistent cloud storage after each epoch—should have been implemented from the beginning; it was only introduced after a broken-off run had lost several hours of training. If the planning had considered the possibility of infrastructure failure rather than assuming that it would succeed, a great deal of time could have been saved.

(3) Giving an honest account of the findings. Some of the results were less satisfactory than had been hoped for: the architectural advantage vanished almost entirely when the cases were fairly compared, the Sydney correction had no effect at all, and gradient boosting turned out to be competitive with both the deep models. It is better to report the results honestly than to present them selectively, and the negative outcomes have important practical consequences for the choice of models in this kind of problem.

(4) Sequencing. A great deal of effort was invested in training the transformer before the baseline models had been finished, under the belief that the advanced model was the focus of the project and that the baselines were merely routine. This order was reversed. The baselines turned out to be the more informative part because, without a competitive gradient boosting result, the closeness of the final comparison would not have been obvious, and the

project would have concluded that a transformer was necessary even though the evidence did not support that. A strong baseline having been established first would have better guided all the following decisions.

(5) The importance of supervision. The two most interesting findings from the project arose because of the supervisory challenge rather than due to independent initiative—the request for standard baselines led to the comparison mentioned above, and the observation that only one model had been tuned gave rise to the main methodological conclusion. At the time both were rather uncomfortable and in each case they contributed to an improvement in the work. The general lesson that can be drawn from this is that a result which supports the hypothesis should be regarded with greater scepticism than one which contradicts it, since it is now when the answer desired is obtained that the temptation to cease investigating is most strong.

REFERENCES

Angelopoulos, A.N. and Bates, S. (2021) A gentle introduction to conformal prediction and distribution-free uncertainty quantification. arXiv preprint arXiv:2107.07511.

Bauer, P., Thorpe, A. and Brunet, G. (2015) The quiet revolution of numerical weather prediction. Nature, 525(7567), pp. 47-55.

Bi, K., Xie, L., Zhang, H., Chen, X., Gu, X. and Tian, Q. (2023) Accurate medium-range global weather forecasting with 3D neural networks. Nature, 619(7970), pp. 533-538.

Box, G.E.P. and Jenkins, G.M. (1970) Time Series Analysis: Forecasting and Control. San Francisco: Holden-Day.

Breiman, L. (2001) Random forests. Machine Learning, 45(1), pp. 5-32.

Challu, C., Olivares, K.G., Oreshkin, B.N., Garza, F., Mergenthaler-Canseco, M. and Dubrawski, A. (2023) NHITS: Neural hierarchical interpolation for time series forecasting. Proceedings of the AAAI Conference on Artificial Intelligence, 37(6), pp. 6989-6997.

Chen, T. and Guestrin, C. (2016) XGBoost: A scalable tree boosting system. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 785-794.

Grinsztajn, L., Oyallon, E. and Varoquaux, G. (2022) Why do tree-based models still outperform deep learning on typical tabular data? Advances in Neural Information Processing Systems, 35, pp. 507-520.

Hersbach, H., Bell, B., Berrisford, P., Hirahara, S., Horanyi, A., Munoz-Sabater, J., et al. (2020) The ERA5 global reanalysis. Quarterly Journal of the Royal Meteorological Society, 146(730), pp. 1999-2049.

Hochreiter, S. and Schmidhuber, J. (1997) Long short-term memory. Neural Computation, 9(8), pp. 1735-1780.

Hong, T. and Fan, S. (2016) Probabilistic electric load forecasting: A tutorial review. International Journal of Forecasting, 32(3), pp. 914-938.

Januschowski, T., Gasthaus, J., Wang, Y., Salinas, D., Flunkert, V., Bohlke-Schneider, M. and Callot, L. (2020) Criteria for classifying forecasting methods. International Journal of Forecasting, 36(1), pp. 167-177.

Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q. and Liu, T.Y. (2017) LightGBM: A highly efficient gradient boosting decision tree. Advances in Neural Information Processing Systems, 30.

Lam, R., Sanchez-Gonzalez, A., Willson, M., Wirnsberger, P., Fortunato, M., Alet, F., et al. (2023) Learning skilful medium-range global weather forecasting. Science, 382(6677), pp. 1416-1421.

Lim, B., Arik, S.O., Loeff, N. and Pfister, T. (2021) Temporal Fusion Transformers for interpretable multi-horizon time series forecasting. International Journal of Forecasting, 37(4), pp. 1748-1764.

Lundberg, S.M. and Lee, S.I. (2017) A unified approach to interpreting model predictions. Advances in Neural Information Processing Systems, 30.

Makridakis, S., Spiliotis, E. and Assimakopoulos, V. (2020) The M4 Competition: 100,000 time series and 61 forecasting methods. International Journal of Forecasting, 36(1), pp. 54-74.

Montero-Manso, P. and Hyndman, R.J. (2021) Principles and algorithms for forecasting groups of time series: Locality and globality. International Journal of Forecasting, 37(4), pp. 1632-1653.

Oreshkin, B.N., Carpov, D., Chapados, N. and Bengio, Y. (2020) N-BEATS: Neural basis expansion analysis for interpretable time series forecasting. International Conference on Learning Representations.

Salinas, D., Flunkert, V., Gasthaus, J. and Januschowski, T. (2020) DeepAR: Probabilistic forecasting with autoregressive recurrent networks. International Journal of Forecasting, 36(3), pp. 1181-1191.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L. and Polosukhin, I. (2017) Attention is all you need. Advances in Neural Information Processing Systems, 30.

Vovk, V., Gammerman, A. and Shafer, G. (2005) Algorithmic Learning in a Random World. New York: Springer.

Zeng, A., Chen, M., Zhang, L. and Xu, Q. (2023) Are Transformers effective for time series forecasting? Proceedings of the AAAI Conference on Artificial Intelligence, 37(9), pp. 11121-11128.

APPENDICES

APPENDIX A: PROJECT PROPOSAL

The approved project proposal is submitted separately as Tharun_Bisai_MSc_Project_Proposal.pdf.

APPENDIX B: PROJECT MANAGEMENT

The work was carried out in five stages. The first of these involved data acquisition and exploratory analysis, this including the cross-city comparison which had been the reason for raising the question about pooling. The second stage consisted of setting up baseline models, both statistical and machine learning ones. The third stage was devoted to training the deep architectures, this stage proving to be the longest because of the problems with GPU availability referred to in Chapter 3. The fourth stage focused on uncertainty quantification and interpretability. The fifth stage corrected the tuning asymmetry that had been identified during supervision and brought the results together.

The supervision meetings had a tangible effect on the work at three places. The first instruction, which was to start with data visualisation and to become familiar with the techniques, laid the foundation. When it was later noticed that the code had not been shared, this caused a more detailed submission of information about the implementation. Most important of all, since it had been observed that only the transformer had been tuned, a search was carried out for all the models because of which the project's main conclusion changed.

The schedule followed is set out below.

Period	**Activity**
Late May 2026	Topic scoping and proposal drafting. Retrieval of ten years of hourly records for the five cities from the Open-Meteo historical API.
12 June 2026	Project proposal approved by the supervisor.
Mid-June 2026	Exploratory data analysis, extended at the supervisor’s request to give more weight to data visualisation. The cross-city comparison produced here raised the pooling question that became the central research question.
Late June 2026	Baseline models, both statistical and machine learning, and the first recurrent network. Initial transformer training.
4 July 2026	Four supervisor requests completed: justification of normalisation, the Sydney hemisphere experiment, justification of the transformer feature roles, and the addition of standard baselines.

Period	Activity
12 July 2026	Subsampled transformer runs found to be undertrained. Decision taken to carry out one clean full data run on a GPU.
17 July 2026	Transformer retraining completed, reaching a test error of 1.247 degrees. This superseded the earlier comparison and reversed the answer to the architecture question.
Late July 2026	Conformal prediction calibration and the explainability analysis. Browser artefact rebuilt and deployed.
Early August 2026	Correction of the tuning asymmetry identified during supervision. Every model given an equivalent hyperparameter search, which changed the project's principal conclusion. Results consolidated and the report written.
11 August 2026	Final submission.

APPENDIX C: ARTEFACT/DATASET

The dataset includes hourly meteorological data for Jena, London, New York, Sydney and Tokyo from the period 2000 to 2009 and was obtained from the Open-Meteo historical API before being made available at kaggle.com/datasets/tharun2431/multi-city-weather-2000-2009.

The code is available in the public repository at https://github.com/tharun2431/weather-forecasting-msc, within the Submission_Code folder. It contains sixteen notebooks, fourteen of these notebooks having their saved cell outputs, four supporting scripts, the full set of result files and the figures used in this report, together with the trained transformer checkpoint so that the reported result can be reproduced without a graphics processor; a README file in the folder explains what each notebook does and specifies which result file supports each of the claims made in this report.

The browser application which is being used carries out neural inference on the client device and can be accessed at tharun2431.github.io/weather-forecasting-msc; it provides live forecasts for all five cities together with conformal prediction intervals. Its source code is held in the same repository, at the top level.

APPENDIX D: SCREENCAST

A screencast demonstrating the artefact and walking through the implementation is available at https://roehamptonprod-my.sharepoint.com/:v:/g/personal/bisait_roehampton_ac_uk/IQDRwSDIKKrMQ4EZVyqveDdKAboH1WGi6GNJfpQd5djL_LU. It is hosted on University of Roehampton OneDrive and access is restricted to members of the University.

APPENDIX E: USE OF ARTIFICIAL INTELLIGENCE

During the drafting of this report, AI applications such as Grammarly were utilized to refine grammar and spelling. Additionally, ChatGPT served as a supplemental resource to troubleshoot programming errors and explore potential debugging strategies. Despite the use of these tools, the overarching design, software implementation, experimentation, data analysis, and final conclusions remain entirely my original work.